

DG INTERNS HUB

WEEK 9 — SOC INTRODUCTION & SETUP

Building a Basic Monitoring Environment with Kali Linux and Snort IDS

Candidate	Md Mohtasim (DG/JULY/CYBER/017)
Module	Security Operations Center (SOC)
Target	Metasploitable 2 (intentionally vulnerable VM, run from its .vmdk hard disk)
Sensor & Traffic Source	Kali Linux running Snort IDS
Environment	Local lab — VirtualBox, host-only network
Primary Tool	Snort (open source IDS)
Submission Date	07-10-2026

1. Introduction

A scanner or a firewall alone does not make an organisation secure. Somebody has to watch what the tools report, decide which events matter, and act on them. That function is what a SOC provides, and the tooling at its heart is a way of turning raw network and host activity into alerts that a person can reason about.

The Week 9 brief asks for the foundation of that picture: understand SOC fundamentals, then build the smallest useful monitoring setup. Snort was chosen because it is free, widely taught, packaged in the Kali repositories, and its rule language is simple enough that every field of a rule can be understood and explained rather than copied. The brief also requires free tools only, which Snort, VirtualBox, Kali Linux and Metasploitable all satisfy.

Metasploitable 2 was chosen as the target because it is a purpose-built, intentionally vulnerable machine that runs a real web server and many network services. That makes it a safe stand-in for a production server: it produces realistic traffic to watch, and no harm is possible because it lives only on an isolated virtual network. Kali Linux plays the part of the machine an attacker or penetration tester would work from, and in this lab it also hosts the Snort sensor.

1.1 Objectives

- Explain what a SOC is, who staffs it, and how SIEM, IDS and IPS differ.
- Prepare a Linux machine for monitoring and complete a baseline configuration (updates, hostname, networking).
- Bring up Metasploitable 2 from its existing hard-disk image and connect it, together with Kali Linux, to an isolated network.
- Install Snort, verify it, and validate its configuration in test mode.
- Understand Snort rule structure and write two custom rules: ICMP and HTTP.
- Generate matching traffic, capture the resulting alerts, and document the evidence.

2. SOC Fundamentals

2.1 What is a Security Operations Center

A Security Operations Center is the team, the processes and the technology responsible for continuously monitoring an organisation's systems, detecting security events, investigating them, and coordinating the response. The key word is *continuously*: attackers do not work office hours, so a SOC is usually staffed around the clock or backed by an on-call arrangement.

It helps to think of a SOC as three things working together. **People** make judgement calls that software cannot. **Process** — playbooks, escalation paths, severity definitions — makes those calls consistent. **Technology** — sensors, log collection, correlation, ticketing — supplies the evidence. A SOC with excellent tools and no process produces alert noise; a SOC with a good process and no visibility is guessing

2.2 What a SOC actually does

- **Monitoring** — watching network traffic, endpoint activity and application logs for signs of abuse.
- **Detection** — turning observations into alerts through signatures, thresholds and correlation logic.
- **Triage and investigation** — deciding whether an alert is a real incident, a false positive, or benign-but-unusual.
- **Response** — containing the problem, supporting eradication and recovery, and recording what happened.
- **Improvement** — tuning rules, closing visibility gaps and feeding lessons back into detection.

2.3 Analyst roles: L1, L2 and L3

Most SOC's split analyst work into tiers so that routine volume is handled quickly while rare, complex problems reach the most experienced people.

Tier	Main responsibility	Typical activities	Escalates when
L1 Analyst (triage)	First look at every alert in the queue	Check the alert against known-benign patterns, gather basic context (source, destination, asset), close false positives, open tickets for real events	The alert looks genuine, or the analyst cannot explain it
L2 Analyst (incident responder)	Deeper investigation of confirmed or likely incidents	Correlate across log sources, examine affected hosts, determine scope, recommend containment, tune noisy rules	The incident is large, novel, or needs specialised analysis
L3 Analyst (senior / hunter)	Advanced analysis and proactive work	Threat hunting, malware and forensic analysis, detection engineering, post-incident review, mentoring L1 and L2	Seldom escalates — owns the hardest cases and the SOC's detection quality

The tiers are a workflow, not a ranking of worth. An L1 analyst who triages quickly and documents clearly is what makes the L2 and L3 time available for work that needs it.

2.4 SIEM, IDS and IPS

These three terms are frequently blurred together. They solve different problems, and the cleanest way to separate them is to ask what each one looks at and what it does about it.

	IDS	IPS	SIEM
Full name	Intrusion Detection System	Intrusion Prevention System	Security Information and Event Management
Primary job	Detect suspicious activity and raise an alert	Detect suspicious activity and block it automatically	Collect, normalise and correlate logs from many sources, then alert and report
Placement	Passive — watches a copy of traffic (span port, tap, or a monitoring interface)	Inline — traffic must pass through it	Central — receives logs from sensors, servers, firewalls and endpoints

Action taken	Alerts only; a human or another system responds	Drops or resets malicious traffic in real time	Correlation alerts, dashboards, investigation search, compliance reports
Risk if it errs	A missed detection (false negative) goes unnoticed	A false positive can block legitimate traffic and cause an outage	Poor log coverage or poor correlation rules produce blind spots or noise
Example	Snort or Suricata in alert-only mode	Snort or Suricata inline mode	Splunk, Elastic Security, Wazuh

2.5 How the three fit together

In a mature SOC the IDS and IPS act as *sensors* and the SIEM acts as the *memory and the brain*. Snort sees a suspicious packet and writes an alert; the alert is forwarded to the SIEM; the SIEM places it next to firewall logs, authentication logs and endpoint events and can recognise that the same source also failed ten logins a minute earlier. That cross-source view is something no single sensor can provide. This week's lab builds only the sensor, but it is built so that its alert output could later be shipped to a SIEM.

2.6 The incident lifecycle in one paragraph

A commonly used model describes four stages: preparation, detection and analysis, containment/eradication/recovery, and post-incident activity. The lab in this report sits squarely in the second stage — it produces the detection that starts everything else. Preparation is what the VM build and rule-writing represent, and the post-incident stage is where tuning decisions, such as those discussed in Section 11, would be made.

3. Lab Design, Scope & Methodology

3.1 Topology

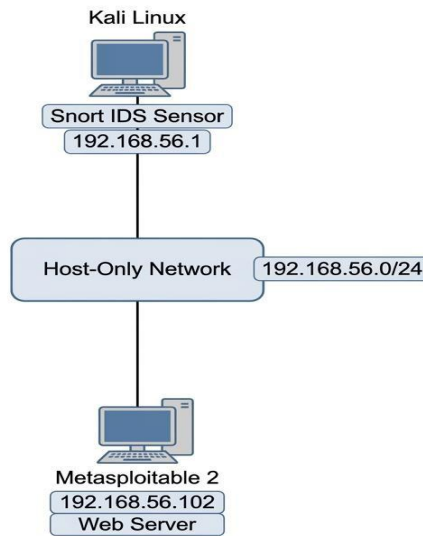


Figure 1 — SOC Monitoring Lab Network Topology

Two machines share one host-only network. Kali Linux sends the test traffic and runs Snort; Metasploitable 2 is the machine being pinged and browsed. Because Snort runs on the same machine that sends the traffic, it sees every packet Kali sends to Metasploitable and every reply that comes back, with no special network settings required. Kali's second adapter is NAT-mode so it can reach the internet for updates and package installs; only the host-only interface is monitored.

3.2 Substitution for Ubuntu Server

The brief calls for Ubuntu Server as the sensor platform. Ubuntu Server was not available in this lab, so Kali Linux was used as the Linux platform for Snort. For the purposes of this task the two are equivalent where it matters: Kali is a Debian-derived distribution, as Ubuntu is, it uses the same apt package manager, and Snort comes from the same Debian packaging with the same configuration layout (`/etc/snort/snort.conf`, `/etc/snort/rules/local.rules`). The installation steps, rule syntax and test procedure in this report therefore apply unchanged.

The difference is architectural rather than technical. In a real SOC the sensor is a dedicated machine, separate from any traffic source. Here one machine plays both roles, so the lab demonstrates the detection loop but not the independence of a standalone sensor. Section 9 records this as a limitation and Section 11 lists moving the sensor onto its own machine as a next step.

3.3 Planned machine specifications

Machine	Role	Specification	Network
Kali Linux (soc-kali01)	Traffic source and Snort sensor	Existing Kali install (VM or host machine), 2 GB RAM or more	NAT adapter for updates; host-only adapter, static 192.168.56.20 as a VM (typically 192.168.56.1 if Kali is the host)
Metasploitable 2	Target	Existing Metasploitable .vmdk attached as the hard disk; Linux 32-bit profile, 512 MB RAM	Single host-only adapter, static 192.168.56.30

ypervisor: Oracle VirtualBox, which is free and meets the brief's free-tools rule. The Kali hostname `soc-kali01` describes the machine's role and is easy to spot in prompts and logs. Addresses are fixed so the test commands in this report are repeatable; if the lab uses different addresses, the actual ones should be substituted throughout

3.4 Methodology

The work followed the same linear style used in earlier weeks: one change at a time, verify, then move on.

3.4.1 Prepare Kali: confirm the VM settings, update packages, set the hostname, configure the host-only network.

3.4.2 Create the Metasploitable VM from its existing hard-disk image and confirm Kali can reach it.

3.4.3 Install Snort on Kali, confirm the version, and validate the configuration with test mode before running anything live.

3.4.4 Write the two custom rules in the local rules file and re-validate.

3.4.5 From a second Kali terminal, generate each kind of traffic against Metasploitable separately, watch for the matching alert, and capture the output as evidence.

3.5 Assumptions and constraints

3.5.1 **Lab-only scope:** all traffic, addresses and alerts originate inside the private host-only network. Nothing in this report was directed at any third-party system.

3.5.2 **Isolation of the target:** Metasploitable is intentionally full of known vulnerabilities and must never be given a NAT or bridged adapter. It is attached to the host-only network only.

3.5.3 **Benign test traffic only:** the tests are a ping and ordinary page requests. No exploitation of Metasploitable is part of this week's task.

3.5.4 **Detection, not prevention:** Snort is run in IDS mode. It reports; it does not block.

3.5.5 **Packaged version:** Snort is installed from the Kali repositories. That is normally the 2.9 series, and the rule syntax in this report is for that series. Snort 3 uses a different configuration layout and slightly different rule options.

3.5.6 **Small traffic sample:** only deliberately generated test traffic was used, so no claims are made about false-positive rates on real networks.

4 Environment Setup

This section covers the virtual machine, system update, hostname and network configuration the brief asks for. Because Kali is already installed, the work here is preparing it and bringing up the target.

4.1 Virtual machine settings

Both machines run in Oracle VirtualBox. Kali has two adapters: Adapter 1 in NAT mode for internet access, and Adapter 2 in host-only mode for the lab network. Metasploitable has a single host-only adapter. The VM list and Kali's network settings were reviewed before starting.

```
(kali@localhost)-[~]
$ VBoxManage list hostonlyifs
Name:                vboxnet0
GUID:                effffff9-dae8-4abf-8000-0a0027000000
DHCP:                Disabled
IPAddress:           192.168.56.1
NetworkMask:         255.255.255.0
IPv6Address:         fe80::800:27ff:fe00:0
IPv6NetworkMaskPrefixLength: 64
HardwareAddress:     0a:00:27:00:00:00
MediumType:          Ethernet
Wireless:            No
Status:              Up
VBoxNetworkName:     HostInterfaceNetworking-vboxnet0
```

Figure 2 — VirtualBox Host-Only Network Adapter Configuration.

4.2 System update

The first action on any Linux system used for monitoring is to bring the package index and installed packages up to date, so later installs resolve against current versions and known fixes are applied. Kali is a rolling distribution, so a full upgrade is used.

```
sudo apt update
sudo apt full-upgrade -y
```

```
(kali@localhost)-[~]
$ sudo apt update
Get:1 https://packages.microsoft.com/repos/code stable InRelease [3,590 B]
Hit:2 http://http.kali.org/kali kali-rolling InRelease
Get:3 https://packages.microsoft.com/repos/code stable/main amd64 Packages [30.2 kB]
Fetched 33.8 kB in 1s (36.0 kB/s)
1473 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Figure 3 — Kali Linux System Update

4.3 Hostname setup

The default hostname was replaced with one that describes the machine's role. Descriptive hostnames matter in a SOC because they are what appears in alerts and logs when an analyst is trying to tell machines apart at speed.

```
sudo hostnamectl set-hostname soc-kali01
hostnamectl
```

The change was confirmed with `hostnamectl`, and the `/etc/hosts` loopback entry was updated so the new name resolves locally. A new terminal was opened so the prompt shows the new name.



Figure 4 — Hostname configured and verified.

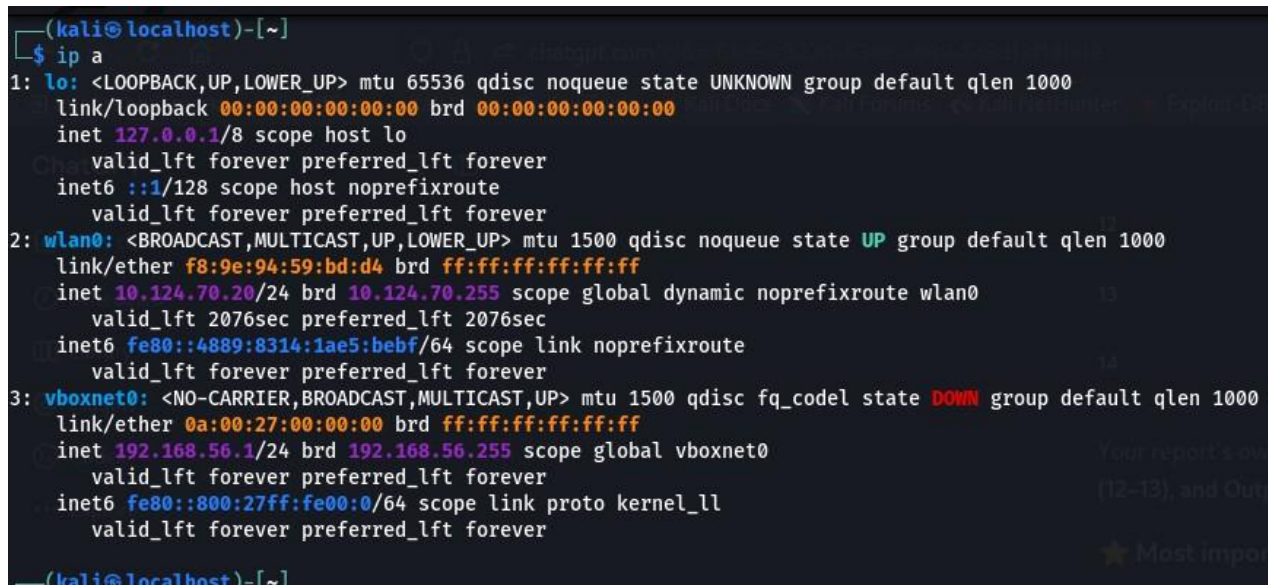
4.4 Network configuration

Kali manages networking with NetworkManager. The NAT adapter keeps its default DHCP configuration. The host-only adapter receives a fixed address so that the lab addresses never change between sessions. The connection name and interface name were first read from the commands below; on a typical Kali VM the NAT interface is `eth0` and the host-only interface is `eth1`, but the names in the actual lab take precedence if they differ.

```
ip a
nmcli con show

# replace the connection name with the one shown for the host-only adapter
sudo nmcli con mod "Wired connection 2" ipv4.method manual ipv4.addresses 192.168.56.20/24
sudo nmcli con up "Wired connection 2"
ip a show eth1
```

If Kali is the host machine running VirtualBox rather than a VM, no change is needed: VirtualBox already creates a host-only interface (normally `vboxnet0`) with the address 192.168.56.1, and that interface is the one Snort will monitor. In that case this step is only a check of the interface name and address.

Figure 5 — Kali Linux network interfaces and IP configuration using `ip a`

4.5 Metasploitable 2 target (from the existing hard disk)

Metasploitable 2 is distributed as a ready-made virtual hard disk rather than an installer, so no operating system installation is needed. A new VM is created and, on the hard-disk step, the option *Use an existing virtual hard disk file* is chosen and the Metasploitable `.vmdk` file is selected. The profile used is Linux (Ubuntu, 32-bit) with 512 MB of RAM. Before the first boot the network adapter is set to host-only and no other adapter is enabled.

```
(kali@localhost:~)~
# VBoxManage list vms
"Metasploitable2" {c3d46ac-4b73-40cd-ad7e-d26e52b612}

(kali@localhost:~)~
# VBoxManage modifyvm "Metasploitable2" --nic1 hostonly --hostonlyadapter1 vboxnet0

(kali@localhost:~)~
# VBoxManage showvminfo "Metasploitable2" | grep -i -A 5 "NIC 1"
NIC 1:      82540EM, Attachment: Host-only Interface 'vboxnet0', Cable connected: on, Trace: off (file: none), Type: 82540EM, Reported speed: 0 Mbps, Boot priority: 0, Promisc Policy: deny, Bandwidth group: none
NIC 2:      disabled
NIC 3:      disabled
NIC 4:      disabled
NIC 5:      disabled
NIC 6:      disabled
```

Figure 6 — Metasploitable 2 Host-Only Network Configuration

The machine boots to a text console login. Its well-known default credentials are published with the image, which is acceptable only because the machine is isolated. A fixed address was then set so the test commands stay stable.

```
# on Metasploitable, after logging in (default msfadmin / msfadmin)
sudo nano /etc/network/interfaces

auto eth0
iface eth0 inet static
address 192.168.56.30
netmask 255.255.255.0

sudo /etc/init.d/networking restart
ifconfig eth0
```

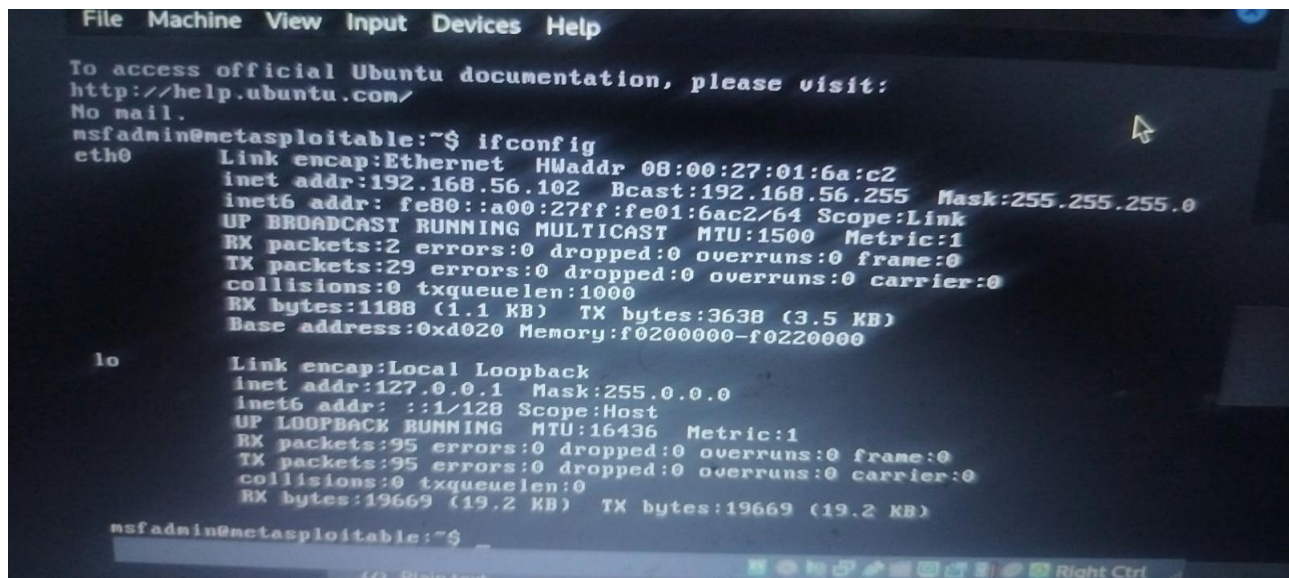


Figure 7 — Metasploitable 2 Network Interface Configuration

4.6 Connectivity check from Kali

Reachability of the target was checked before any Snort work began, so that a later missing alert could not be blamed on basic connectivity. These requests are made before Snort is running, so they are not part of the detection tests.

```
ping -c 3 192.168.56.30          # Metasploitable
curl -I http://192.168.56.30 # Metasploitable web server answers on port 80
```

The `curl -I` request confirms that the Apache web server on Metasploitable is listening, which the HTTP test in Section 8 depends on.

```
(kali@localhost)-[~]
$ ping -c 4 192.168.56.102
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data.
64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=0.470 ms
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=0.560 ms
64 bytes from 192.168.56.102: icmp_seq=3 ttl=64 time=0.484 ms
64 bytes from 192.168.56.102: icmp_seq=4 ttl=64 time=0.598 ms

--- 192.168.56.102 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3082ms
rtt min/avg/max/mdev = 0.470/0.528/0.598/0.052 ms
```

Figure 8 — Connectivity Test Between Kali Linux and Metasploitable 2

```
(kali@localhost)-[~]
$ curl -I http://192.168.56.102
HTTP/1.1 200 OK
Date: Wed, 07 Oct 2026 11:11:29 GMT
Server: Apache/2.2.8 (Ubuntu) DAV/2
X-Powered-By: PHP/5.2.4-2ubuntu5.10
Content-Type: text/html
```

Figure 8(b) – Metasploitable 2 Host-Only Network Configuration

5 Installing Snort

5.1 Installation

Snort is available directly from the Kali repositories, so no source build was required. During installation the package asks which interface to listen on and what the local address range is; the answers used were Kali's host-only interface (eth1, or vboxnet0 if Kali is the host) and 192.168.56.0/24. These values populate `HOME_NET`, a variable the rules rely on to distinguish 'inside' from 'outside' traffic.

```
sudo apt install snort -y
```

```
(kali@localhost)-[~]
$ sudo apt install snort -y
snort is already the newest version (3.12.2.0-0kali1).
The following packages were automatically installed and are no longer required:
  idle-python3.13  libpoppler147  libpython3.13-testsuite  libxmlsec1-nss1  python3.13-dev  python3.13-examples  python3.13-minimal
  libgsoap-2.8.139  libpython3.13-dev  libxmlsec1-1  python3.13  python3.13-doc  python3.13-full  python3.13-venv
Use 'sudo apt autoremove' to remove them.

Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1473
```

Figure 9 — Snort Installation Verification

5.2 Verification

Installation was verified in two ways: asking the binary for its version, and inspecting the configuration it will load.

```
snort -V
grep -n "ipvar HOME_NET" /etc/snort/snort.conf
grep -n "local.rules" /etc/snort/snort.conf
```

The last command matters for Section 7: it confirms that the main configuration file includes `local.rules`. A custom rule written into a file that is never loaded would silently do nothing, which is one of the most common beginner mistakes.

```
(kali@localhost)-[~]
$ snort -V

--> Snort++ <*-
Version 3.12.2.0
By Martin Roesch & The Snort Team
http://snort.org/contact#team
Copyright (C) 2014-2026 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using DAQ version 3.0.24
Using libpcap version 1.10.6 (64-bit time_t, with TPACKET_V3)
Using LuaJIT version 2.1.1761786044
Using LZMA version 5.8.3
Using OpenSSL 3.6.3 9 Jun 2026
Using PCRE2 version 10.46 2025-08-27
Using ZLIB version 1.3.2
```

Figure 10 — Snort Version Verification

5.3 Running in test mode

Test mode (`-T`) parses the whole configuration and every rule, then exits without monitoring anything. It is the safe way to check that a configuration is valid before pointing Snort at live traffic.

```
sudo snort -T -c /etc/snort/snort.conf -i eth1
```

A healthy run ends with a message stating that Snort successfully validated the configuration. Any syntax problem in a rule or the config is reported with a file name and line number instead, which makes test mode the first thing to run after every rule edit. If test mode stops on a rule file that is not present on the system, the matching `include` line in `snort.conf` can be commented out, since only `local.rules` is needed for this lab.

A practical note on checksums. Because Snort runs on the same machine that sends the test traffic, outgoing packets can carry unfinished checksums on a virtual network card (the card fills them in later). Snort may treat those packets as corrupt and silently ignore them. Adding `-k none` to the live run commands in Section 8 turns off checksum verification for the lab and avoids missed alerts.

```
(kali@localhost)-[~]
$ sudo snort -T

-----
o")~  Snort++ 3.12.2.0
-----
-----
pcap DAQ configured to passive.

Snort successfully validated the configuration (with 0 warnings).
o")~  Snort exiting
```

Figure 11 — Snort Configuration Test and Validation

6 Understanding Snort Rule Structure

A Snort rule is a single line made of two parts: a **header** that says which traffic to look at, and **options** in parentheses that say what to look for inside it. Reading a rule left to right tells you exactly what it will and will not catch.

```
action proto src_ip src_port direction dst_ip dst_port (options)
alert icmp any any -> $HOME_NET any (msg:"..."; sid:...;
```

7 Custom Rules

Two rules were written, each aimed at one of the traffic types named in the brief. They live in the local rules file so they are kept separate from the packaged rulesets and survive package updates.

```
sudo nano /etc/snort/rules/local.rules
```

7.1 Rule 1 — ICMP echo request (ping) detection

```
alert icmp any any -> $HOME_NET any (msg:"LAB-ICMP Echo request (ping)
detected"; \ itype:8; classtype:misc-activity; sid:1000001; rev:1;)
```

The trailing backslash continues the rule onto the next line for readability; Snort treats it as one rule, and the lines can equally be joined into one.

Reading the rule. It alerts on ICMP traffic from any source to any address inside the protected range. The `itype:8` option narrows it to echo requests only, so the reply packets travelling back are not counted twice. In a real network, a burst of echo requests from one source across many addresses is a classic sign of a ping sweep, an early reconnaissance step; a single ping on its own is usually benign, which is why this rule would normally be paired with a threshold in production.

7.2 Rule 2 — HTTP traffic detection

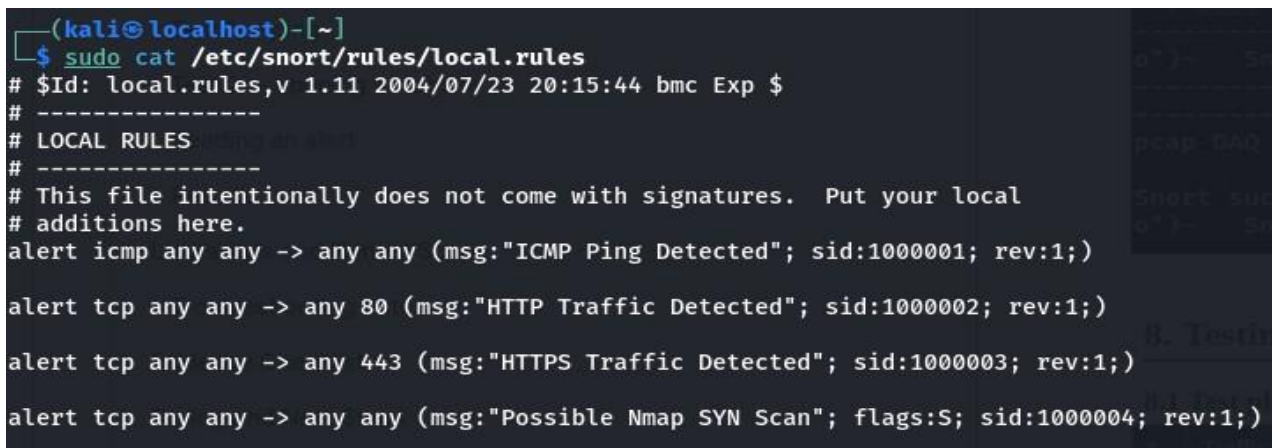
```
alert tcp any any -> $HOME_NET 80 (msg:"LAB-HTTP GET request to web server"; \
flow:to_server,established; content:"GET "; depth:4; \
classtype:web-application-activity; sid:1000002; rev:1;)
```

Reading the rule. It watches TCP traffic going to port 80 on any protected host. The `flow` option restricts matches to packets travelling client-to-server on an established connection, and `content:"GET "` with `depth:4` requires those four characters at the very start of the payload, which is how an HTTP GET request begins. Using the raw payload rather than HTTP-specific buffers keeps the rule portable across Snort configurations, at the cost of being less precise than a rule built on the HTTP preprocessor. That trade-off is revisited in Section 9.

7.3 Applying and validating the rules

After saving the file, the configuration was validated again. This step confirms both that the syntax is correct and that the rules are loaded.

```
sudo snort -T -c /etc/snort/snort.conf -i eth1
```



```
(kali@localhost)-[~]
$ sudo cat /etc/snort/rules/local.rules
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# -----
# LOCAL RULES
# -----
# This file intentionally does not come with signatures. Put your local
# additions here.
alert icmp any any -> any any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)

alert tcp any any -> any 80 (msg:"HTTP Traffic Detected"; sid:1000002; rev:1;)

alert tcp any any -> any 443 (msg:"HTTPS Traffic Detected"; sid:1000003; rev:1;)

alert tcp any any -> any any (msg:"Possible Nmap SYN Scan"; flags:S; sid:1000004; rev:1;)
```

Figure 12 — Custom Snort Rules in *local.rules*

```
(kali@localhost)-[~]
$ sudo snort -T
-----
o")~  Snort++ 3.12.2.0
-----
-----
pcap DAQ configured to passive.

Snort successfully validated the configuration (with 0 warnings).
o")~  Snort exiting
```

Figure 13 — Snort Configuration Validation with Custom Rules

8 Testing and Alert Capture

8.1 Test plan

#	Traffic generated	From	Command or action	Rule expected to fire
1	ICMP echo request to the target	Kali	ping -c 4 192.168.56.30	1000001
2	HTTP GET request to the target	Kali	Open http://192.168.56.30 in Firefox	1000002
3	Control: no matching traffic	—	Leave Snort running with no test traffic	None

Test 3 is a control. A detection that only ever fires is not evidence of a working rule — it could be firing on everything. Watching Snort stay quiet while idle gives the alerts in tests 1 and 2 meaning.

8.2 Starting the sensor

Metasploitable already runs an Apache web server, so nothing needs installing on the target. Two terminal windows were used on Kali: one to run Snort and one to send test traffic. Snort was started in the first, in console alert mode on the monitored interface.

```
sudo snort -A console -q -k none -c /etc/snort/snort.conf -i eth1
```

In console mode alerts print to the terminal as they happen, which makes the cause-and-effect between a test action and an alert easy to see. To keep a permanent record, a second run writes alerts to a file instead:

```
sudo snort -A fast -q -k none -l /var/log/snort -c /etc/snort/snort.conf -i eth1
sudo tail -f /var/log/snort/alert
```

8.3 Test 1 — ping

From the second Kali terminal, four echo requests were sent to Metasploitable while Snort ran in the first. Each echo request should produce one alert referencing rule 1000001, with Kali as the source and

Metasploitable as the destination. The echo replies travelling back are type 0 and are deliberately not matched. If the ping succeeds but no alert appears, the interface name and the checksum note in Section 5.3 are the first things to check.

```
(kali@localhost)-[~]
$ ping -c 4 192.168.56.102
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data.
64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=0.293 ms
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=0.458 ms
64 bytes from 192.168.56.102: icmp_seq=3 ttl=64 time=0.306 ms
64 bytes from 192.168.56.102: icmp_seq=4 ttl=64 time=0.233 ms

--- 192.168.56.102 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3076ms
rtt min/avg/max/mdev = 0.233/0.322/0.458/0.082 ms
```

8.4 Test 2 — opening a website

Metasploitable's web server was opened from Firefox on Kali. Its default page lists the vulnerable web applications installed on the target. Loading it generates at least one GET request, and browsers often request a favicon too, so more than one alert for rule 1000002 is normal. Only the front page was loaded; none of the applications listed on it were used.

As an optional extra, `nmap -sn 192.168.56.0/24` from Kali sends echo requests across the whole segment and should trigger rule 1000001 repeatedly. That is a small demonstration of why the rule needs a threshold, as Section 11 discusses.